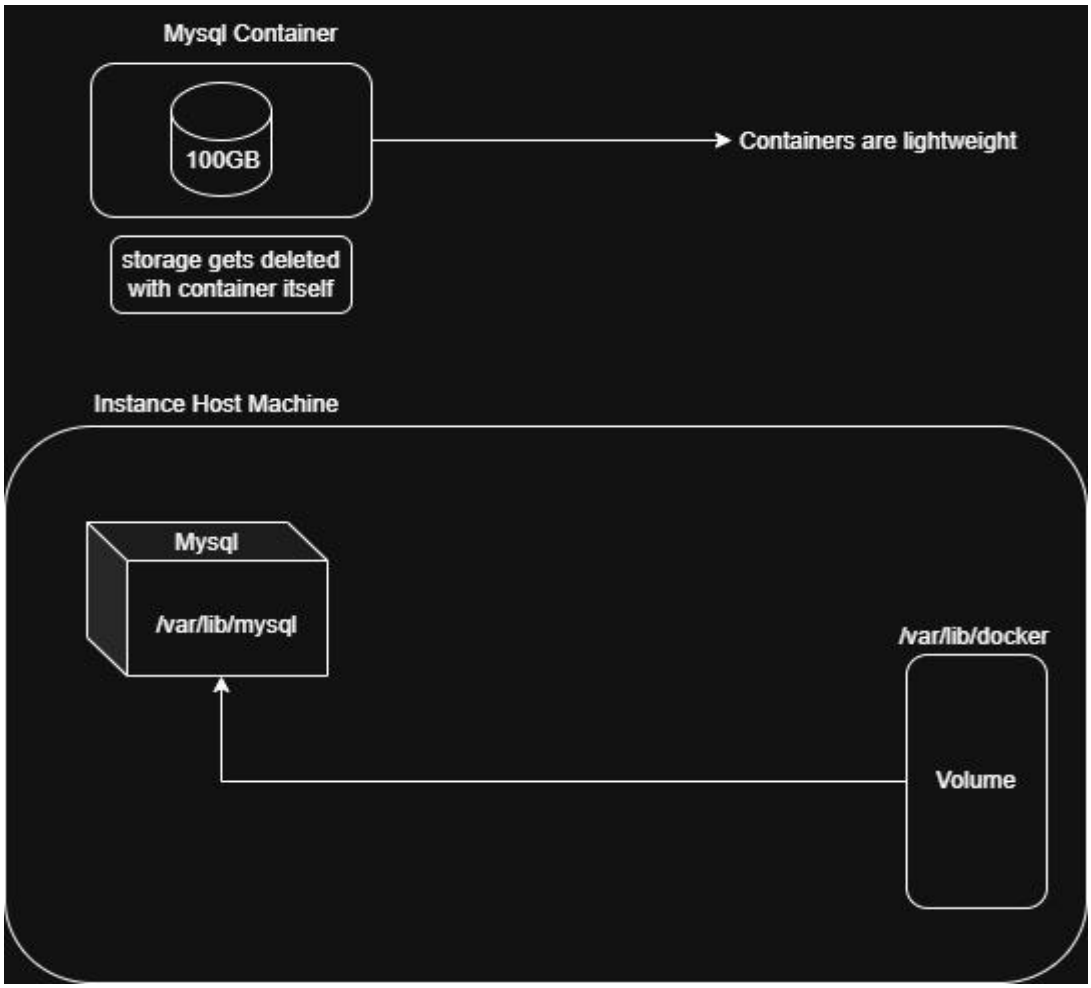
PERSISTENT STORAGE WITH DOCKER VOLUMES

What is Docker Volume

1. A Docker volume is used to store data permanently outside the container's writable layer.
2. This means that even if the container stops, is removed, or recreated, the data stored in the volume remains safe.
3. we can allocate a single volume to multiple containers to share the data
4. we cannot allocate or create a volume from the existing container only we can allocate while creating.
5. volume will not be included when you update an image.



TYPES OF VOLUMES

1. Named volume
2. Bind mounts volume
3. Anonymous Volume

NAMED VOLUMES

A volume which is directly managed by Docker, stored in Docker's internal storage (usually /var/lib/docker/volumes/).

COMMANDS

Create a instance & attach a volume

1. docker volume create my_volume
2. ls /var/lib/docker/volume/
3. docker run -d -p 8081:80 –name nginx -v my_volume:/usr/share/nginx/html/ nginx
4. <public_ip:8081>
5. ls /var/lib/docker/volumes/

Delete a instance & reattach this volume to another instance

1. docker run -d -p 81:80 –name httpd -v my_volume:/usr/local/apache2/htdocs/ httpd
2. <public_ip:81>
3. cd /var/lib/docker/volumes/
4. ls

BIND MOUNTS

Maps a specific file or directory from the host machine into the container.

1. Directly uses host filesystem
2. Full control over file location
3. Changes reflect immediately on both sides
4. Less portable (depends on host path)

PRACTICAL

1. mkdir /root/mywebsite
2. cd mywebsite
3. vim index.html
   <h1> Welcome from Host Machine </h1>
4. docker run -d -p 8081:80 --name nginx_container -v /root/mywebsite:/usr/share/nginx/html nginx
5. <public_ip:8081>
6. cd mywebsite
7. Vim index.html
   <h1> Welcome to Bind Mount </h1>
8. <public_ip:8081>

ANONYMOUS VOLUME

1. Data is shared between containers.
2. If one container changes files, the other container can see them.
3. Useful for backup containers or shared storage.
4. --volumes-from copies volume mounting information from another container.

CREATE CONTAINER-1

1.docker run -it --name container1 -v /volume ubuntu /bin/bash

1. cd /volume
2. touch file{1..10}
3. exit

SHARE VOLUME WITH CONTAINER-2

1.docker run -it --name container2 --volumes-from container1 ubuntu /bin/bash

1. cd /volume
2. ls

Container2 is using the same volume created by Container1.

HOW TO DELETE VOLUMES

1. docker volume delete my_volume

HOW TO CREATE MYSQL CONTAINER

1. docker volume create mysql-data
2. docker volume ls
3. docker run -d --name mysql-container -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root123 -v mysql-data:/var/lib/mysql mysql:latest
4. docker exec -it mysql-container /bin/bash
   mysql -u root -h localhost -proot123
5. ls /var/lib/docker/volumes/mysql-data/
   {You get all the files of mysql-container}

Note:

1. /var/lib/mysql is the default location where MySQL stores databases.
